

SportsBook Project Study Guide

This guide assumes you know literally zero about the project. Read it like a story first, then use the interview Q&A section to revise.

1. One-Line Explanation

SportsBook is a campus sports management web app where IIIT Delhi users can:

- sign up and log in
- book courts/facilities
- issue sports equipment
- generate QR codes for bookings and gym attendance
- see live occupancy
- see live match scores
- find other players
- create team invites
- request or offer coaching/mentorship

Very simple version:

It is like a digital sports office. Instead of going to a physical desk and asking, “Is a badminton court free?”, the app checks the database, books it, updates availability, and shows everyone the new status.

2. Tech Stack

Layer	Used In Project	Simple Meaning
Frontend	Next.js 15, React 19, TypeScript	The screens/buttons/forms users see
Routing	Next.js App Router	Folder-based pages like <code>/dashboard</code> , <code>/book-court</code>
Backend logic	Next.js Server Actions	Server-side functions called by UI
Database	PostgreSQL	Permanent storage
ORM	Prisma	Type-safe-ish bridge between code and database
Auth	JWT with <code>jose</code> , password hashing with <code>bcryptjs</code>	Login/session system
Cache/rate limit	Upstash Redis	Counts requests and blocks abuse
Real-time	Socket.io client, external socket server expected	Pushes live updates
UI styling	Tailwind CSS, Framer Motion, lucide-react	Design, animation, icons
Charts	Recharts	Occupancy analytics charts
File upload	Cloudinary	Uploaded image storage
Email	Nodemailer	Password reset emails

3. Project Map

```
Sportsbook-TS-main
|
|-- src/app/           Pages/routes
|   |-- page.tsx       Home page
|   |-- login/page.tsx Login + signup
|   |-- dashboard/page.tsx User dashboard + active booking QR
|   |-- book-court/page.tsx Main booking form
|   |-- rto/page.tsx    Real-time occupancy + analytics
|   |-- live-scores/page.tsx Live match scoreboard
|   |-- playersearch/  Matchmaking, teams, invites, assistance
```

```

|   |-- profile/page.tsx      Edit user profile
|   |-- sg/page.tsx          Self-growth/gym QR + gym history
|   |-- past-bookings/page.tsx Booking history
|   |-- forgot-password/     Password reset request
|   |-- reset-password/      Set new password
|
|-- src/actions/             Server-side backend functions
|   |-- auth.ts              Signup, login, logout, reset password
|   |-- bookings.ts          Book/extend/return/expire booking
|   |-- sports.ts            Sports list + analytics
|   |-- gym.ts               Gym scan and gym stats
|   |-- matches.ts           Live score CRUD
|   |-- invites.ts           Team invites
|   |-- applications.ts      Mentor/coaching requests
|   |-- files.ts             Upload/preview files
|
|-- src/contexts/            Global client state
|   |-- AuthContext.tsx      Current logged-in user
|   |-- SportsContext.tsx    Cached list of sports
|   |-- SocketContext.tsx    Socket connection
|   |-- ThemeContext.tsx     Dark/light mode
|   |-- ToastContext.tsx     Toast messages
|
|-- src/lib/                 Shared helpers
|   |-- prisma.ts            Database client
|   |-- auth-config.ts       JWT secret + token verification
|   |-- auth-utils.ts        requireUser, ensureAdmin, access checks
|   |-- rate-limiter.ts      Redis sliding-window limiter
|   |-- validation.ts        Input validation
|
|-- prisma/schema.prisma     Database design
|-- prisma/reset-db.js       Seed/reset script

```

4. Whole App Visual

flowchart TD

```

User[Student/User] --> UI[Next.js Pages]
UI --> Contexts[React Contexts]
UI --> Actions[Server Actions]
Contexts --> UI
Actions --> Auth[Auth Helpers]
Actions --> Prisma[Prisma Client]
Prisma --> DB[(PostgreSQL)]
Actions --> Redis[(Upstash Redis)]
Actions --> SocketServer[External Socket Server]
SocketServer --> SocketClient[Socket.io Client]
SocketClient --> UI
Actions --> Cloudinary[Cloudinary]
Actions --> Email[Nodemailer SMTP]

```

Simple story:

1. User clicks something on a page.
2. Page calls a server action.
3. Server action checks login and validates data.
4. Server action updates PostgreSQL through Prisma.
5. If needed, it tells the socket server.
6. Socket server tells connected browsers to refresh live data.

5. Database Models

Think of each model as a table.

erDiagram

```
User ||--o{ Booking : makes
User ||--o{ GymLog : has
Sport ||--o{ Equipment : owns
Booking ||--o{ BookingEquipment : uses
Equipment ||--o{ BookingEquipment : linked
```

```
User {
    string id
    string email
    string password
    string name
    string phone
    string rollNumber
    string[] sportsExperience
    string role
}
```

```
Sport {
    string id
    string name
    int numberOfCourts
    int courtsInUse
    json courtData
    int maxCapacity
    int numPlayers
}
```

```
Equipment {
    string id
    string name
    int total
    int inUse
    string sportId
}
```

```
Booking {
    string id
    string userId
    string sportName
    int numberOfPlayers
    string startTime
    string endTime
    string date
    string status
    string courtNo
    boolean scanned
}
```

```
Match {
    string id
    string sportName
    string team1
    string team2
}
```

```

        string score1
        string score2
        string status
    }

    Invite {
        string id
        string sport
        string venue
        string date
        string time
        string email
        boolean show
    }

    GuideApplication {
        string id
        string email
        string option
        string sportName
        string level
        boolean resolved
    }
}

```

Important Tables Explained Like You Are 6

Table	Meaning	Example
User	A person using the app	Kunal, Admin, student
Sport	A playable facility type	Badminton, Gym, Swimming
Equipment	Items linked to a sport	Racket, Shuttle, Ball
Booking	A user's reservation	Kunal booked Badminton Court 1
BookingEquipment	Equipment used in a booking	Booking X issued 2 rackets
Match	Live score item	Team A vs Team B
Invite	Public team invitation	Need players for football at 6 PM
GuideApplication	Mentorship/coaching request	Need beginner tennis help
GymLog	Gym entry/exit record	Checked in at 6 PM, out at 7 PM

6. Core User Flows

Flow A: Signup

```

sequenceDiagram
    participant U as User
    participant LoginPage as /login page
    participant AuthAction as createAction()
    participant DB as PostgreSQL
    participant Cookie as Browser Cookie

    U->>LoginPage: fills signup form
    LoginPage->>LoginPage: checks @iiitd.ac.in email
    LoginPage->>AuthAction: createAction(data)
    AuthAction->>AuthAction: validate email/password/name
    AuthAction->>DB: check if email exists
    AuthAction->>AuthAction: bcrypt.hash(password)
    AuthAction->>DB: create user
    AuthAction->>Cookie: set JWT session cookie

```

```
AuthAction-->>LoginPage: success
LoginPage-->>U: redirect to dashboard
```

Key interview answer:

Passwords are not stored directly. They are hashed using bcrypt. After signup/login, a JWT is created and stored in an HTTP-only cookie called `session`.

Flow B: Login

flowchart LR

```
A[User enters email/password] --> B[login action]
B --> C[Redis rate limit: max 10 attempts per 15 min]
C --> D[Find user by email]
D --> E[Compare password with bcrypt]
E --> F[Create signed JWT]
F --> G[Set HTTP-only session cookie]
G --> H[Dashboard]
```

Important security points:

- Login attempts are rate-limited.
- Password comparison uses `bcrypt.compare`.
- JWT is signed using `JWT_SECRET`.
- Cookie is `httpOnly`, so client-side JavaScript cannot read it.

Flow C: Book A Court

This is the most important feature.

sequenceDiagram

```
participant U as User
participant Page as /book-court
participant Action as secureBooking()
participant DB as PostgreSQL
participant Socket as Socket Server
participant Other as Other Users

U->>Page: chooses sport, court, duration, equipment
Page->>Action: secureBooking(payload)
Action->>Action: ensure user is self/admin
Action->>DB: start transaction
Action->>DB: lock User row
Action->>DB: check existing pending/active/returned booking
Action->>DB: lock Sport row
Action->>DB: check capacity/court availability
Action->>DB: update sport occupancy
Action->>DB: check equipment availability
Action->>DB: create booking and equipment links
Action->>DB: add bookingId into QR detail
Action->>DB: commit transaction
Action->>Socket: notify availability changed
Socket->>Other: refresh live occupancy
```

Simple explanation:

The app first locks the relevant database rows, like saying “nobody else touch this court while I am checking it.” Then it checks if the court and equipment are free. If yes, it books them. If no, it rejects the request.

Why transaction matters:

- Without a transaction, two users could book the same court at the same time.

- With a transaction and FOR UPDATE, the database handles one booking at a time for that critical row.

Flow D: Court-Based Sport vs Capacity-Based Sport

flowchart TD

```

A[Selected sport] --> B{Has maxCapacity?}
B -- Yes --> C[Capacity-based facility]
C --> D[Example: Gym or Swimming]
C --> E[Check numPlayers + requested players <= maxCapacity]

B -- No --> F[Court-based sport]
F --> G[Example: Badminton, Tennis, Squash]
F --> H[Check selected courtData flag]
H --> I[0 = free, 1 = booked]

```

Type	Examples	Availability Logic
Court-based	Badminton, Squash, Table Tennis, Tennis	Specific court must be free
Capacity-based	Gym, Swimming	Total players must not exceed capacity

Flow E: Dashboard Active Booking

flowchart TD

```

A[Dashboard loads] --> B[getBookings for current user]
B --> C{Any pending/active/returned booking?}
C -- No --> D[Show quick actions]
C -- Yes --> E[Show active booking card]
E --> F[Generate QR using booking.qrDetail]
E --> G[Show countdown]
G --> H{Time over?}
H -- Yes --> I[requestReturn booking]
H -- No --> G

```

Booking statuses used by backend:

Status	Meaning
pending	Booking created, waiting for QR/admin scan
active	Booking is running
returned	User ended session; admin approval may still be pending
expired	Session is over/closed

Flow F: Return / End Session

flowchart LR

```

A[User clicks End Session] --> B[requestReturn]
B --> C[Lock booking]
C --> D[Lock sport]
D --> E[Decrease courtsInUse / numPlayers]
E --> F[Mark booking as returned]
F --> G[Notify socket server]

```

Important:

The code releases sport occupancy during `requestReturn`, but equipment `inUse` is decremented in `expireBooking`, not clearly in `requestReturn`. This is an interview-risk area. If asked, say:

The intended flow seems to be: user requests return, admin later confirms/cleans up. But in this repository alone, equipment release is not fully obvious from the user app, so I would verify the admin portal flow.

Flow G: Real-Time Occupancy

sequenceDiagram

```
participant Page as Browser Page
participant SocketClient as Socket.io Client
participant Action as Server Action
participant SocketServer as Socket Server
participant DB as DB

Page->>SocketClient: join sport room
Action->>DB: update booking/sport
Action->>SocketServer: POST /notify-update
SocketServer->>SocketClient: availability_changed
SocketClient->>Page: call refreshSports(true)
Page->>Action: getSports()
Action->>DB: fetch latest sports
```

Where this appears:

- `SocketContext.tsx`: creates socket connection
- `useSocket.ts`: joins a sport room and listens for events
- `bookings.ts`: notifies socket server after booking changes
- `rto/page.tsx`: refreshes occupancy when socket event arrives
- `book-court/page.tsx`: refreshes availability when someone else books

7. Pages Explained

Page	Route	What It Does
Home	/	Landing page with sports/features
Login	/login	Login/signup with IIITD email check
Dashboard	/dashboard	Shows active booking, QR, countdown, quick actions
Book Court	/book-court	Main booking form
Live Scores	/live-scores	Shows live/finished/upcoming matches
RTO	/rto	Real-time occupancy, court view, analytics
Player Search	/playersearch	Matchmaking, team making, assistance, invites
Profile	/profile	Edit user details and sports experience
SG	/sg	Gym/self-growth dashboard and gym QR
Past Bookings	/past-bookings	Booking history
Forgot Password	/forgot-password	Sends reset email
Reset Password	/reset-password	Sets new password using token
Smart Booking	/smartbooking	Demo/static availability search, not fully backend-connected

8. Server Actions Cheat Sheet

`auth.ts`

Function	Purpose
<code>createAccount</code>	Validate signup, hash password, create user, create session
<code>login</code>	Rate-limit, verify password, create session
<code>logout</code>	Delete session cookie
<code>getCurrentUser</code>	Read session cookie and return user

Function	Purpose
<code>getUsers</code>	Return public user list for matchmaking
<code>updateUser</code>	Allow self/admin to update profile
<code>requestPasswordReset</code>	Create reset token and send email
<code>resetPassword</code>	Verify reset token and set new password

`bookings.ts`

Function	Purpose
<code>secureBooking</code>	Main booking function with transaction, capacity check, equipment check
<code>createBooking</code>	Simpler booking function; less complete than <code>secureBooking</code>
<code>getBookings</code>	Fetch bookings; user gets own bookings, admin can fetch more
<code>extendBooking</code>	Extend time, max total duration 4 hours
<code>requestReturn</code>	User ends session; frees court/capacity and marks returned
<code>expireBooking</code>	Marks expired and decrements equipment in-use
<code>updateBooking</code>	Admin/self update booking fields
<code>deleteBooking</code>	Delete booking

Other Actions

File	Important Functions
<code>sports.ts</code>	<code>getSports</code> , <code>getSport</code> , <code>getSportAnalytics</code>
<code>gym.ts</code>	<code>handleGymScan</code> , <code>getGymStats</code>
<code>matches.ts</code>	<code>getMatches</code> , <code>createMatch</code> , <code>updateMatch</code> , <code>deleteMatch</code>
<code>invites.ts</code>	<code>createInvite</code> , <code>getInvites</code> , <code>editInvite</code> , <code>deleteInvite</code>
<code>applications.ts</code>	<code>createApplication</code> , <code>getApplications</code> , <code>deleteApplication</code>
<code>files.ts</code>	<code>uploadFile</code> , <code>getFilePreview</code>

9. React Contexts

Context	Stores	Why Needed
<code>AuthContext</code>	current user, login/logout, loading	Avoid fetching user separately on every page
<code>SportsContext</code>	sports list, refresh function	Shared sports/court availability data
<code>SocketContext</code>	socket object, connected status	One socket connection for the whole app
<code>ThemeContext</code>	dark/light theme	App-wide theme toggle
<code>ToastContext</code>	toast messages	Reusable success/error messages

Visual:

flowchart TD

```

RootLayout --> ToastProvider
ToastProvider --> AuthProvider
AuthProvider --> SocketProvider
SocketProvider --> SportsProvider
SportsProvider --> ThemeProvider
ThemeProvider --> Navbar
ThemeProvider --> PageContent

```

10. Security Design

Good Security Choices

Feature	Where	Why Good
Password hashing	auth.ts	Plain passwords are never stored
JWT session cookie	auth.ts, auth-config.ts	Stateless login
HTTP-only cookie	createSession	Client JS cannot read token
Strong JWT secret validation	auth-config.ts	Rejects weak/default secrets
Login rate limit	login	Slows brute force attempts
Password reset token hashing	requestPasswordReset	DB stores hashed token, not raw token
Generic reset response	PASSWORD_RESET_RESPONSE	Prevents email enumeration
Input validation	validation.ts	Blocks missing/bad data
Access helpers	auth-utils.ts	Centralized self/admin checks
Redis global/IP limiter	middleware.ts	Protects whole app from spam

Middleware Flow

flowchart TD

```
A[Request arrives] --> B{POST/PUT/PATCH/DELETE?}
B -- Yes --> C{Server Action or allowed origin?}
C -- No --> D[403 Forbidden]
C -- Yes --> E[Rate limit by IP]
B -- No --> E
E --> F{IP limit OK?}
F -- No --> G[429 Too Many Requests]
F -- Yes --> H[Global rate limit]
H --> I{Global limit OK?}
I -- No --> J[429 Too Many Requests]
I -- Yes --> K{Admin route?}
K -- Yes --> L[Verify JWT role Admin]
K -- No --> M[Continue]
L --> M
```

11. Database Concurrency

Most interviewers like this part.

Problem:

What if two users click “Book Court 1” at the same time?

Project answer:

`secureBooking` uses `prisma.$transaction` and raw SQL `FOR UPDATE`.

Meaning:

The database locks the user/sport row while booking is being checked and updated. The second request must wait, so both cannot successfully book the same court.

Important code behavior:

- Lock user row.
- Check whether this user already has an active/pending/returned booking.
- Lock sport row.
- Check court or capacity availability.
- Update sport counters.
- Check equipment inventory.
- Create booking.
- Link equipment to booking.

12. QR Codes

There are two QR concepts.

QR	Used For	Data Inside
Booking QR	Dashboard active booking	qrDetail JSON containing booking/user/sport details
Gym QR	Navbar and SG page	User ID

The actual image is generated using:

<https://api.qrserver.com/v1/create-qr-code/>

Interview answer:

The app does not draw QR codes itself. It creates a URL to a QR generation service and passes encoded data.

13. Player Search / Team Making

Matchmaking

Uses `getUsers()` and filters users by:

- name search
- sport
- skill level

Sports skill data is stored in `User.sportsExperience` as strings like:

Badminton:Beginner

Tennis:Expert

Team Making

flowchart LR

```
A[Choose sport] --> B[Load users]
B --> C[Filter users who play that sport]
C --> D[Show eligible players]
D --> E[Create invite]
E --> F[Invite appears in current invites]
```

Assistance / Mentorship

Users can:

- apply as mentor
- request coaching assistance
- choose sport, level, description, available days, time

Data goes into `GuideApplication`.

14. Occupancy Analytics

The RTO page has three tabs:

Tab	Meaning
Live Stats	Current players, occupancy %, equipment usage
Schedule	Placeholder schedule area
Analytics	Weekly attendance and average load by hour

Analytics source:

- `getSportAnalytics(sportName)` fetches bookings for the last 7 dates.
- It builds:
 - `weeklyAttendance`: total players per day
 - `peakHours`: average users in 2-hour slots

```

flowchart TD
    A[Selected sport] --> B[getSportAnalytics]
    B --> C[Fetch last 7 days bookings]
    C --> D[Group by day]
    C --> E[Group by time slot]
    D --> F[Weekly Attendance Bar Chart]
    E --> G[Average Load Bar Chart]
  
```

15. Seed Data

`prisma/reset-db.js` is intended to:

- refuse reset unless `ALLOW_DB_RESET=true`
- require strong seed passwords from env
- clear tables
- create sports
- create users/admins
- create completed past bookings for analytics

Seeded sports include:

- Badminton
- Squash
- Table Tennis
- Tennis
- Swimming
- Gym

16. Environment Variables

The README lists these categories:

Variable Group	Purpose
<code>DATABASE_URL</code>	PostgreSQL connection
<code>JWT_SECRET</code>	Signing JWT session tokens
<code>UPSTASH_REDIS_REST_URL</code> , <code>UPSTASH_REDIS_REST_TOKEN</code>	Redis rate limiting
<code>NEXT_PUBLIC_SOCKET_URL</code> , <code>SOCKET_INTERNAL_SECRET</code>	Socket server communication
Cloudinary vars	Image upload
SMTP vars	Password reset email

Important interview line:

The app intentionally fails if critical secrets are missing or weak, using `requireServerEnv`.

17. Current Bugs / Weak Spots

Be honest if asked. These are not insults to the project; they are improvement points.

Area	Issue	Impact
Dependency install	In this environment, <code>npm install</code> failed because a package download was forbidden by registry policy	I could not run a full build here

Area	Issue	Impact
Socket server	README mentions <code>socket-servers</code> , but this zip does not include that folder	Real-time needs external server not present in package
Booking status mismatch	Backend validation allows <code>pending</code> , <code>active</code> , <code>returned</code> , <code>expired</code> ; seed uses <code>completed</code> ; live scores UI expects <code>upcoming</code> , but backend match validation allows only <code>live</code> , <code>finished</code>	Some UI filters may never show expected states
Past bookings equipment	Past bookings page checks <code>booking.issuedEquipments</code> , but backend returns <code>equipments</code> relation	Equipment may show as “No equipment issued” even when records exist
Equipment release	<code>requestReturn</code> releases court/capacity, but equipment <code>inUse</code> decrement is mainly in <code>expireBooking</code>	Equipment counts can stay wrong if admin/expiry flow is incomplete
Admin flow	User app protects admin-only match mutations, but admin portal is not included	Some flows depend on a separate system
Smart booking	<code>/smartbooking</code> uses sample slots, not real backend availability	Demo page, not production booking logic
Too much <code>any</code>	Many actions/components use <code>any</code>	Reduces TypeScript safety
Debug logs/alerts	Many <code>console.log</code> , <code>alert</code> , <code>confirm</code> calls	Fine for prototype, less polished for production
QR service dependency	QR image depends on third-party service	If service is down, QR display fails
Client-side email domain check	Signup page checks <code>@iiiitd.ac.in</code> , but server <code>createAccount</code> only validates email format	A direct server-action call could bypass domain restriction unless middleware/other layer handles it

18. Interview Answers

Q1. What is this project?

SportsBook is a campus sports management platform. It lets students book courts, issue equipment, track occupancy, see live scores, find players, form teams, and manage gym attendance using QR codes.

Q2. What is your role in the project?

Good answer:

I worked on the user-facing sports booking platform. The key parts are the booking flow, authentication, real-time occupancy refresh, Prisma data models, and server actions that safely update court/equipment availability.

Q3. Why Next.js?

Next.js gives routing, frontend, and backend-style server actions in one project. For this app, UI pages call server actions directly, so booking and authentication logic can stay close to the frontend while still running securely on the server.

Q4. Why Prisma?

Prisma makes database operations easier and safer than writing SQL everywhere. The schema defines models like User, Sport, Booking, and Equipment, and Prisma generates a client to query/update them.

Q5. How do you prevent double booking?

The main function is `secureBooking`. It runs inside a database transaction and uses row locks with `FOR UPDATE`. It locks the sport row, checks if the court/capacity is available, updates occupancy, checks equipment, and creates

the booking atomically.

Q6. What happens when a user books a court?

The UI builds a payload with user, sport, players, duration, selected court, and equipment. It calls `secureBooking`. The server validates access, starts a transaction, checks existing user bookings, checks sport capacity/court status, updates sport and equipment counts, creates the booking, embeds the booking ID into QR data, commits, then notifies the socket server.

Q7. How does real-time update work?

Pages connect to the socket server through `SocketContext`. They join a sport room using the custom `useSocket` hook. After a booking changes, the server action calls the socket server's notify endpoint. The socket server emits an event, and the page refreshes sports data.

Q8. How is authentication implemented?

On signup/login, password is hashed or checked using `bcrypt`. A JWT is signed with `jose` and stored in an HTTP-only cookie named `session`. Server helpers read and verify this cookie to get the current user.

Q9. What is Redis used for?

Redis is used for sliding-window rate limiting. The middleware limits requests per IP and globally. Login and password reset also have separate rate limits.

Q10. What are the main database relationships?

A User has many Bookings and GymLogs. A Sport has many Equipment items. A Booking can use many Equipment items through BookingEquipment. Match, Invite, and GuideApplication are separate feature tables.

Q11. What is courtData?

It is a JSON field in Sport. For court sports, it stores strings like `Court 1:0`, where 0 means free and 1 means occupied. This lets the app show and update per-court availability.

Q12. Why are there both `courtsInUse` and `courtData`?

`courtData` tells exactly which court is booked. `courtsInUse` gives a quick count for occupancy percentage and availability display.

Q13. What is the difference between court-based and capacity-based facilities?

Court-based sports need a specific free court. Capacity-based facilities like Gym/Swimming only care whether total players exceed max capacity.

Q14. How does password reset work?

The user requests reset by email. The server creates a random token, stores its SHA-256 hash and expiry in the database, emails the raw token link, and later verifies the hash before allowing password change.

Q15. What would you improve?

Strong answer:

I would first align status enums across frontend/backend/seed data, make equipment return flow explicit, include or document the socket server, remove demo-only smart booking or connect it to real availability, replace broad any types, and add tests for concurrent booking.

19. Feature-by-Feature Revision

Feature	Main Page	Main Action	Database Tables
Signup/Login	/login	createAccount, login	User
Book Court	/book-court	secureBooking	Booking, Sport, Equipment, BookingEquipment
Dashboard QR	/dashboard	getBookings, requestReturn, extendBooking	Booking
Occupancy	/rto	getSports, getSportAnalytics	Sport, Booking, Equipment
Live scores	/live-scores	getMatches	Match
Player search	/playersearch	getUsers	User
Team invites	/playersearch	createInvite, getInvites	Invite
Coaching help	/playersearch	createApplication	GuideApplication
Gym	/sg	getGymStats, handleGymScan	GymLog, User
Profile	/profile	updateUser	User

20. 2-Minute Project Pitch

SportsBook is a full-stack campus sports management platform built with Next.js, TypeScript, Prisma, PostgreSQL, Redis, and Socket.io. The core feature is safe sports booking: a student can select a sport, court, duration, players, and equipment, and the backend creates a booking only if the facility and equipment are available.

The most important technical part is the `secureBooking` server action. It uses a database transaction and row locking to prevent double booking. It updates court occupancy, capacity, and equipment usage atomically, then notifies a socket server so other users see real-time availability updates.

Authentication uses bcrypt for password hashing and JWT session cookies. Redis is used for sliding-window rate limiting at both middleware level and sensitive actions like login/password reset. The app also includes live scores, occupancy analytics, player matchmaking, team invites, coaching requests, and gym QR attendance.

If improving it, I would standardize status values, fully verify equipment return flow, add tests for concurrent booking, reduce any, and include the socket server with the repository.

21. Super Short Mental Model

Remember this:

```

Pages = what user sees
Actions = backend work
Prisma = talks to database
PostgreSQL = stores truth
Redis = counts/rate-limits requests
Socket = tells everyone when data changed
Contexts = share frontend state

```

22. What To Say If You Get Stuck

If interviewer asks a very deep question and you are unsure:

The core design intent is clear: booking state lives in PostgreSQL, protected by server-side actions and transactions. Real-time UI is eventually refreshed using socket events. I would verify that specific edge case by tracing the corresponding server action and database update.

This sounds honest and technical.

23. Final Interview Checklist

Before interview, make sure you can explain:

- What problem SportsBook solves
- Why Next.js server actions are used
- Database tables and relationships
- Full booking flow
- Why transaction + row lock prevents double booking
- Auth flow: bcrypt + JWT cookie
- Redis rate limiting
- Socket real-time refresh
- Difference between court-based and capacity-based sports
- Known bugs and improvements

If you know only one function deeply, know this:

`secureBooking()`

It is the heart of the project.